

PHASE 2

Requirements Analysis

Chapter 3: functional, non-functional, hardware, software and user requirements

FITBUDDY • AI FITNESS PLAN GENERATOR • PROJECT REPORT

Chapter 3: Requirements Analysis

This chapter states what FitBuddy must do (functional requirements), how well it must do it (non-functional requirements) and what is needed to build and run it.

3.1 Functional Requirements

Table 3.1: Functional Requirements

ID	Requirement	Description
FR1	User input	The system shall accept name, user ID, age, weight, fitness goal and preferred intensity through a web form.
FR2	Input validation	The system shall validate all inputs on the server and reject missing or invalid values with a clear message.
FR3	Workout generation	The system shall generate a personalised 7-day workout plan using a Gemini model.
FR4	Nutrition generation	The system shall generate a nutrition/recovery tip using a Gemini Flash model.
FR5	Plan storage	The system shall store the user record, the original plan and the nutrition tip in the database.
FR6	Feedback submission	The system shall allow a user to submit written feedback about a generated plan.
FR7	Revised plan generation	The system shall send the original plan, feedback and profile to Gemini and obtain a revised plan.
FR8	Original plan preservation	The system shall keep the original plan unchanged and store the revised plan separately.
FR9	User retrieval	The system shall retrieve a stored user and associated plans by user ID.
FR10	Admin dashboard	The system shall let authorised administrators view users, original plans, feedback, tips and revised plans.
FR11	Search / filter	The dashboard shall support searching or filtering of user records. [Verify against implemented behaviour]
FR12	API documentation	The system shall expose interactive API documentation at /docs.

3.2 Non-Functional Requirements

Table 3.2: Non-Functional Requirements

Attribute	Requirement
Performance	Pages other than AI-dependent operations should load quickly; AI-dependent operations are bounded by Gemini API latency, and the interface should show a progress indication while waiting.
Reliability	Failures of the Gemini API or database must be handled gracefully, returning an understandable message without corrupting stored data.
Security	API keys must be kept in environment variables and used only on the server; input must be validated; administrative views must be access-restricted.
Maintainability	Code is divided into modules with single responsibilities (routing, schemas, database, generators); model names are configurable.
Usability	The interface should be simple, use clear labels and present plans in a readable, day-by-day layout.
Scalability	The design should allow migration to a server database such as PostgreSQL by changing the connection URL and, where needed, minor ORM adjustments.
Availability	The application depends on network access to the Gemini API; local features (stored data viewing) should remain available when the AI service is unreachable.
Responsiveness	The layout should adapt to desktop, tablet and mobile screen widths.

3.3 Hardware Requirements

The following are reasonable development and demonstration requirements. They are indicative and not mandatory; the application is lightweight because the computationally expensive AI processing is performed by the external Gemini service.

Table 3.3: Hardware Requirements

Component	Recommended Specification
Processor	Dual-core 2.0 GHz or better (any modern laptop CPU)
Memory (RAM)	4 GB minimum; 8 GB recommended for comfortable use of the editor, browser and server together
Storage	1 GB free disk space (SQLite database, virtual environment and project files)
Network	Stable internet connection for Gemini API requests
Display	Standard 1366 x 768 or higher; mobile screen for responsive testing (optional)

3.4 Software Requirements

Table 3.4: Software Requirements

Software	Purpose
Python 3.x [Insert version used]	Programming language for the backend
FastAPI	Web framework for routes, validation integration and API documentation
Uvicorn	ASGI server used to run the application
SQLite	Embedded relational database
SQLAlchemy	Object-relational mapper for database access
Pydantic	Data validation and schema definitions
google-genai	Official Google Gen AI SDK for calling Gemini models
Web browser	Access to the interface and Swagger UI (any modern browser)
Visual Studio Code	Source code editor
Git / GitHub	Version control and code hosting

Table 3.5 relates the technologies to the layers of the system.

Table 3.5: Technology Stack by Layer

Layer	Technologies
Frontend	HTML, CSS, JavaScript, Jinja2 templates, responsive web design
Backend	Python, FastAPI, Uvicorn, Pydantic
Database	SQLite, SQLAlchemy ORM
AI	Google Gemini API, google-genai SDK; a Gemini model for workout generation and Gemini Flash for nutrition/recovery generation
Development tools	Visual Studio Code, Git/GitHub, FastAPI Swagger UI, Python virtual environment

The project uses the current google-genai SDK. The deprecated google-generativeai package is not used. Because model availability and naming change over time, model identifiers are treated as configuration values (Appendix B) rather than constants embedded in the code.

3.5 User Requirements

End users need a modern web browser, an internet connection and the information requested by the form. They do not require technical knowledge. Users should read the health disclaimer and consult a professional if they have medical concerns.

Administrators need the authorised means of accessing the dashboard route, and an understanding that stored data includes personal fitness information which must be handled responsibly. Deployers additionally need a Google API key, the ability to set environment variables and a Python environment in which the dependencies are installed. Table 3.6 lists the input fields collected from the user.

Table 3.6: User Input Fields

Field	Type	Description	Validation (indicative)
Name	Text	Display name of the user	Required, non-empty
User ID	Text / identifier	Identifier linking user, plans and feedback	Required, unique per user [verify]
Age	Integer	Age in years	Required, numeric, within a sensible range
Weight	Number	Body weight (unit as shown on the form)	Required, numeric, positive
Fitness goal	Text / selection	For example fat loss, muscle gain, endurance	Required
Intensity	Selection	Preferred workout intensity (for example low, moderate, high)	Required, from permitted values

[Exact ranges and permitted values are defined in the validation schema; the entries above are indicative. [Insert exact validation rules from schemas.py]]